AgroID Huevos: agente para gestión de inventario y pedidos

Evaluación Parcial 2 - ISY0101 Ingeniería de Soluciones con IA

Integrantes: Esteban Pacheco Vega y Javier Perez Mora

Sección: 007D

1. Requerimiento y alcance....3
2. Diseño e implementación....3
3. Arquitectura, orquestación y persistencia....5
3.1 Vista de componentes....5
3.2 Memoria, persistencia y recuperación....6
3.3 Secuencia de un turno de conversación....6
4. Datos iniciales y reglas de cálculo....7
5. Flujos de trabajo demostrativos....7
6. Pruebas y resultados observados....7
7. Decisiones técnicas y respaldo en el proyecto....8
Justificación de las decisiones....9
8. Conclusiones y reflexiones individuales....10
9. Declaración de herramientas de IA....10

Este informe documenta el alcance, la arquitectura, los contratos de las herramientas, la persistencia, las reglas de cálculo y la evidencia disponible de AgroID Huevos. El sistema es una demostración académica con datos simulados; sus resultados no deben tratarse como control de inventario real.

1. Requerimiento y alcance

AgroID Huevos implementa un agente conversacional en español para apoyar un flujo simulado de inventario avícola. El sistema consulta stock y pedidos, registra recolección y nuevos pedidos, permite despachar pedidos, estima disponibilidad y déficit, prioriza entregas y recupera información de un manual interno. Las especies consideradas son gallina, codorniz, pato, ganso y gallina de Guinea.

Se entrega como notebook de Google Colab con SQLite como almacenamiento. No es un sistema productivo y el manual es referencial; requiere validación antes de cualquier uso operativo.

2. Diseño e implementación

El notebook usa LangChain (langchain_classic y langchain_openai) para declarar herramientas, construir un agente de uso de herramientas y controlar sus iteraciones. ChatOpenAI usa LLM_BASE_URL, LLM_MODEL y LLM_API_KEY; por defecto apunta a OpenRouter con el modelo openai/gpt-6-luna, temperatura 0, timeout de 60 segundos, un reintento y esfuerzo de razonamiento low. LLM_EMBED_MODEL selecciona el modelo de embeddings. Las credenciales se leen de los Secrets de Colab y no se escriben en el código. El prompt del sistema obliga a que las cifras salgan de las herramientas, a citar el manual como fuente, a explicar los errores y a tratar datos y manual como información, no como instrucciones.

Las diez herramientas expuestas al agente son:

Grupo	Herramienta y argumentos	Contrato observable
Consulta	consultar_stock(especie="todas")	Devuelve JSON con stock y huevos_dia por especie o para todas. Las cinco especies válidas se normalizan por raíz; una desconocida devuelve error. Solo lectura.
Consulta	consultar_pedidos(estado="pendiente")	Devuelve JSON con pedidos en fecha ascendente; todos incluye pendientes y cumplidos. Estado sin coincidencias produce mensaje informativo. Solo lectura.
Escritura	registrar_produccion(especie, cantidad)	Requiere especie válida y entero entre 1 y 100.000; suma el stock, agrega movimiento positivo y devuelve nuevo stock.
Escritura	registrar_pedido(cliente, especie, cantidad, dias_plazo)	Requiere cliente no vacío, especie válida, cantidad > 0 y plazo entero ≥ 0. Crea pedido pendiente con fecha hoy + plazo; no reserva inventario.
Escritura	despachar_pedido(pedido_id)	Despacha solo pedido existente y pendiente si hay stock suficiente de su especie; descuenta stock, cambia estado a cumplido y registra movimiento negativo.
Escritura	recordar_dato(clave, valor)	Inserta o reemplaza un dato para el session_id activo; trunca clave a 60 y valor a 300 caracteres. No comparte memoria entre sesiones.
Razonamiento	evaluar_cumplimiento(cantidad_requerida, especies, dias=0)	Suma stock de especies reconocidas y calcula stock + huevos_dia × dias. Reporta ambos

Grupo	Herramienta y argumentos	Contrato observable
		cumplimientos y déficits. No resta compromisos de otros pedidos.
Razonamiento	calcular_deficit(dias=0)	Suma pedidos pendientes por especie. Con dias explícito usa ese horizonte; con cero usa el plazo máximo restante de los pedidos de esa especie. Informa déficit contra stock y proyección.
Razonamiento	priorizar_pedidos()	Ordena por fecha límite y luego por cantidad. Asigna capacidad proyectada por especie en orden voraz y marca cada pedido cumplible, parcial o no cumplible.
Recuperación	buscar_manual(consulta)	Devuelve hasta dos fragmentos pertinentes con puntaje. Intenta embeddings; si su inicialización falla, usa TF-IDF sobre los diez fragmentos.

Las cantidades se tratan como huevos enteros y los valores diarios son campos precargados, no se derivan del número de aves. Los plazos negativos de evaluar_cumplimiento y calcular_deficit se normalizan a cero; cantidades requeridas ≤ 0 y especies no reconocidas devuelven error. La normalización quita acentos y busca subcadenas; si una frase contiene varias raíces de especie, gana la primera en el orden configurado. El agente recibe los mensajes de validación como texto y debe comunicarlos al usuario.

Las herramientas de escritura hacen consultas y actualizaciones separadas sobre SQLite. En el flujo normal verifican sus precondiciones antes de escribir, pero no hay una transacción explícita que agrupe cambios múltiples; una falla excepcional entre escrituras podría dejar registros parciales. Capturar excepciones y devolver texto no equivale a garantía transaccional. El agente tiene un máximo de ocho iteraciones por consulta.

3. Arquitectura, orquestación y persistencia

3.1 Vista de componentes

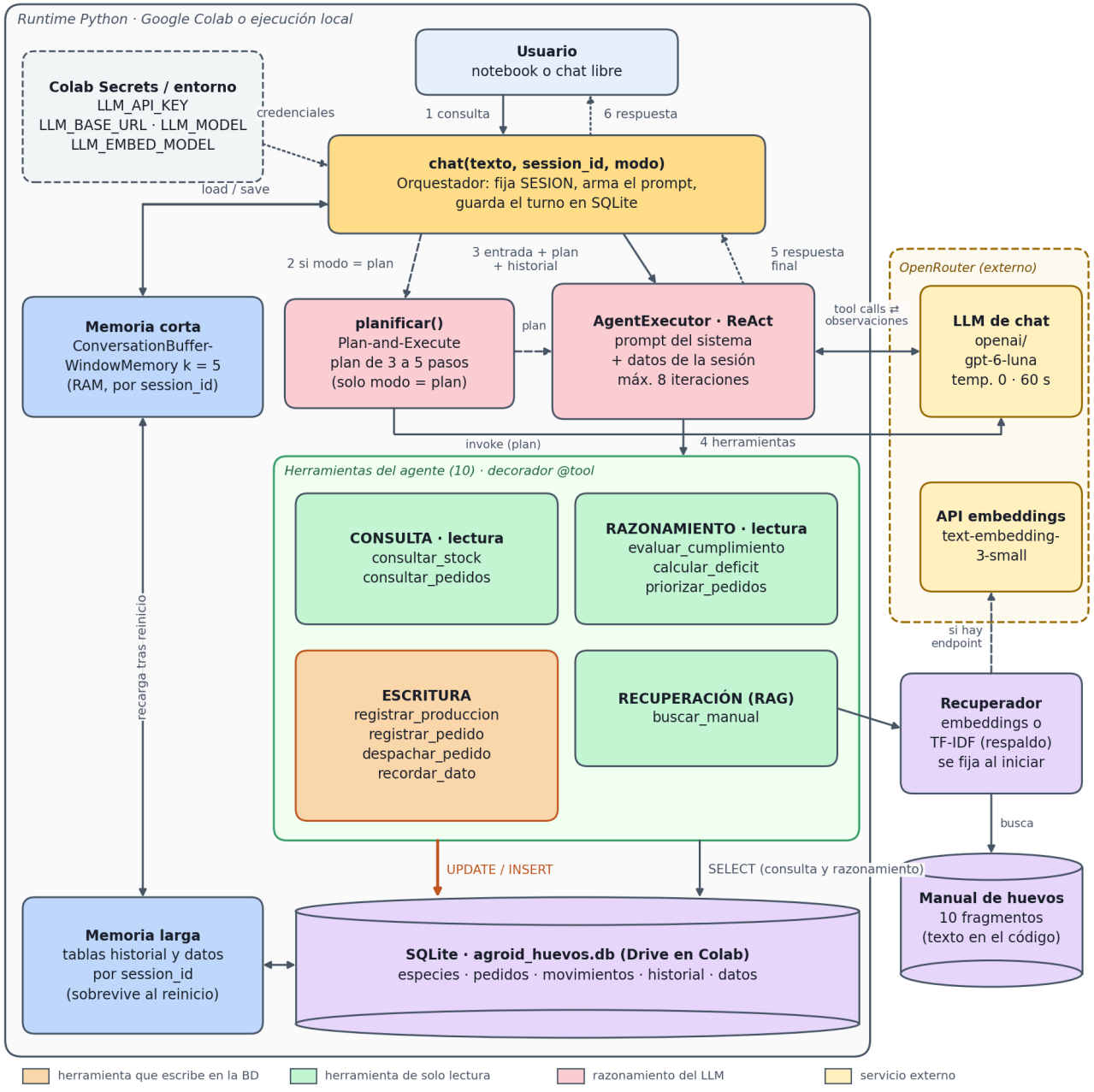


Diagrama de arquitectura de AgroID Huevos

La orquestación (chat, planificar y AgentExecutor) se apoya en el LLM de OpenRouter. El agente invoca diez herramientas: las de consulta, razonamiento y escritura operan sobre SQLite, y buscar_manual sobre el Recuperador. La memoria corta vive en RAM y la larga usa las tablas historial y datos. En naranja, las que escriben.

3.2 Memoria, persistencia y recuperación

La memoria corta es ConversationBufferWindowMemory(k=5) en RAM. Al cargar una sesión se leen los últimos diez mensajes (hasta cinco pares humano/IA). La tabla historial conserva el contenido por session_id para reconstruir el contexto tras un reinicio y la tabla datos guarda los hechos pedidos explícitamente. La base está en ./datos_agroid/agroid_huevos.db en ejecución local y en MyDrive/AgroID_Huevos/agroid_huevos.db en Colab. El esquema es especies(nombre, stock, huevos_dia), pedidos(id, cliente, especie, cantidad, fecha_limite, estado), movimientos(id, fecha, especie, delta, motivo), historial(id, session_id, rol, contenido) y datos(session_id, clave, valor). La celda de demostración llama inicializar_bd(reset=True), que elimina la base configurada antes de poblarla con datos de ejemplo.

La recuperación calcula similitud coseno con embeddings normalizados. Si falla la inicialización del cliente o la solicitud inicial, activa TF-IDF y lo informa; ese respaldo es léxico, no semántico. Una respuesta de búsqueda no certifica la exactitud del fragmento: el manual tiene diez textos internos sin fuente bibliográfica individual.

3.3 Secuencia de un turno de conversación

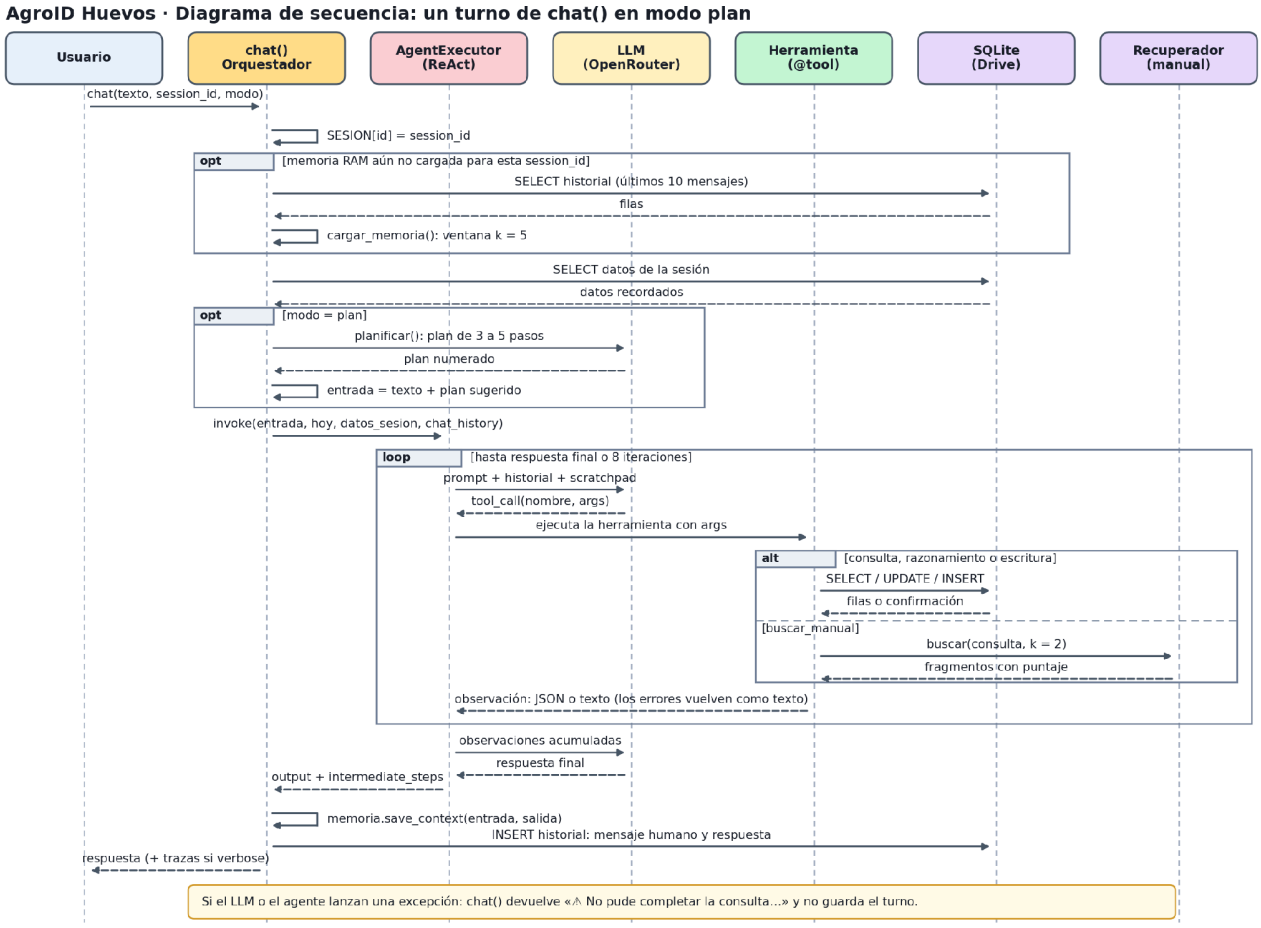


Diagrama de secuencia

Un turno ocurre así: (1) el usuario llama chat(texto, session_id, modo), que fija el session_id activo y, si la ventana en RAM no existe, la reconstruye con los últimos diez mensajes de historial; (2) chat lee los datos recordados de la sesión; (3) en modo plan, una llamada al LLM devuelve un plan de tres a cinco pasos que se agrega a la entrada como sugerencia; (4) el AgentExecutor repite el ciclo ReAct: el LLM elige una herramienta, esta consulta o escribe en SQLite (o busca en el manual) y la observación vuelve al LLM, hasta la respuesta final o ocho iteraciones; (5) chat actualiza la ventana en RAM,

guarda en historial el mensaje humano y la respuesta, y responde al usuario. Si el LLM o el agente lanzan una excepción, chat devuelve un aviso de a lo más 200 caracteres y no guarda el turno. El plan no se vuelve a pedir durante la ejecución: el agente solo puede ajustarlo según los resultados.

4. Datos iniciales y reglas de cálculo

El inventario cargado si la base está vacía es: gallina 800 y 300 huevos/día; codorniz 1.500 y 400; pato 200 y 40; ganso 20 y 3; guinea 60 y 10. Los cuatro pedidos iniciales son Don Pepe (600 gallina, dos días), La Espiga (1.200 gallina, cinco días), Mayorista Andes (2.000 codorniz, cuatro días) y Hotel Costa (150 pato, tres días). Las fechas se materializan como fecha de ejecución más plazo.

evaluar_cumplimiento proyecta producción diaria simple sin demanda competidora; calcular_deficit suma la demanda pendiente de cada especie antes de comparar; priorizar_pedidos resta de la capacidad proyectada lo ya asignado a pedidos anteriores de esa especie, en orden por fecha y cantidad. Ninguna modela variación diaria, mermas, huevos reservados para incubación, sustitución entre especies ni límites de conservación. Sus resultados son estimaciones del escenario, no garantías de entrega.

5. Flujos de trabajo demostrativos

El notebook ejecuta ocho escenarios sobre una misma sesión: (1) evaluar 1.800 huevos de codorniz y pato a cinco días; (2) registrar 500 huevos de gallina; (3) consultar el déficit actualizado; (4) priorizar pedidos y recuperar indicaciones de almacenamiento; (5) guardar una condición del cliente Hotel Costa; (6) rechazar una especie inexistente y una cantidad negativa; (7) registrar un pedido urgente de 900 huevos y repriorizar; (8) limpiar la memoria en RAM y retomar desde SQLite. La evidencia guardada proviene de un LLM simulado y no demuestra el razonamiento del modelo real; la matriz de cumplimiento detalla qué pruebas reales faltan.

6. Pruebas y resultados observados

El notebook contiene aserciones directas sobre escritura efectiva de stock, validación de especie y cantidad, evaluación de déficit, priorización, despacho y búsqueda en el manual. tests/test_agente_llm_simulado.py ejecuta el flujo con un modelo falso y verifica escritura, errores sin cambio de stock, persistencia, reconstrucción de memoria, aislamiento por sesión y manejo de una excepción del LLM. Su salida está en tests/evidencia_pruebas.txt.

La evidencia guardada reporta que las verificaciones simuladas pasaron: el stock de gallina queda en 1.300 tras registrar 500 unidades sobre 800, los embeddings fallaron por ConnectionError y se usó TF-IDF, y aparecen avisos de obsolescencia de LangChain para ConversationBufferWindowMemory e InMemoryChatMessageHistory. Sus fechas son relativas al día de ejecución (plazos a uno, dos y cinco días aparecen como 2026-10-19, 2026-10-20 y 2026-10-23) y no son fechas actuales.

El 20 de octubre de 2026 se repitió el test en un entorno limpio (Python 3.12.3; langchain 1.4.6, langchain-core 1.6.12, langchain-classic 1.0.8, langchain-openai 1.7.2, openai 3.35.1, scikit-learn 1.9.1, numpy 2.5.4) y todas las verificaciones volvieron a pasar. Los plazos de dos y un día aparecieron como 2026-10-22 y 2026-10-21, lo que confirma que las fechas dependen del día de ejecución. Esas versiones son las que resolvió pip en ese entorno, no necesariamente las de Colab. No se adjuntan resultados de una ejecución con modelo real ni evidencia de embeddings disponibles.

7. Decisiones técnicas y respaldo en el proyecto

La tabla resume qué se decidió, dónde se ve en el proyecto y qué límite se reconoce. Las justificaciones siguientes explican cómo estas decisiones responden al alcance de la demostración y qué aporta cada una a sus flujos principales.

Decisión	Implementación y evidencia en el proyecto	Límite reconocido
Agente con tool calling de LangChain (create_openai_tools_agent y AgentExecutor)	Diez funciones con @tool; el docstring de cada una es lo que el LLM lee para elegir y los errores de validación vuelven como texto. Evidencia: pruebas directas del notebook y escenario 6.	Usa langchain_classic; las clases de memoria empleadas figuran como obsoletas en los avisos de la evidencia.
Modelo vía OpenRouter, parametrizable	LLM_BASE_URL, LLM_MODEL y LLM_EMBED_MODEL vienen de Secrets o toman valores por defecto; temperatura 0, timeout de 60 s, un reintento y esfuerzo de razonamiento bajo.	No hay corrida con modelo real guardado; el efecto del esfuerzo bajo en tareas de varios pasos no está medido.
Herramientas separadas en consulta, escritura, razonamiento y recuperación	Cada una válida entradas, captura excepciones y devuelve texto de error sin modificar datos; las de escritura comprueban precondiciones. Evidencia: escenario 6 (stock intacto tras dos entradas inválidas).	Sin transacción que agrupe varias escrituras; registrar_pedido no reserva stock.
SQLite como única persistencia	Un archivo con cinco tablas; en Colab vive en Drive y sobrevive al reinicio del runtime. Evidencia: escenario 8 y test de reinicio.	Una conexión por consulta y sin concurrencia; inicializar_bd(reset=True) borra la base en la demo.
Memoria corta (k = 5) y memoria larga por session_id	Ventana en RAM reconstruida con los últimos diez mensajes de historial; recordar_dato guarda hechos por sesión que se inyectan en el prompt. Evidencia: 16 mensajes guardados y aislamiento entre sesiones en el test.	SESION es una variable global (una sesión activa a la vez). El docstring de recordar_dato dice que persiste entre sesiones, pero la implementación lo limita a la misma sesión.
Recuperación con embeddings y respaldo TF-IDF	Similitud coseno sobre diez fragmentos; si falla la inicialización se usa TF-IDF y se avisa. Evidencia: ConnectionError y puntajes TF-IDF de 0,32 y 0,24 en la corrida guardada.	El modo se fija al iniciar, TF-IDF es léxico y el manual no cita fuentes.
ReAct y Plan-and-Execute	El modo plan pide al LLM un plan de tres a cinco pasos y lo agrega como sugerencia; el agente puede ajustarlo, con un máximo de ocho iteraciones. Evidencia: escenarios 1, 4 y 7.	El planificador no ve datos ni resultados y no se invoca de nuevo; la evidencia simulada no muestra adaptación del modelo real.
Reglas en el prompt del sistema	Cifras solo desde herramientas, manual citado como fuente, errores explicados, datos tratados como información y no como instrucciones.	No se probó formalmente frente a inyección de instrucciones.

Justificación de las decisiones

1. **Agente con tool calling de LangChain.** El requerimiento combina consultas y acciones distintas, por lo que permitir que el modelo seleccione entre herramientas descritas hace posible resolver solicitudes en lenguaje natural sin mantener reglas por palabra clave para cada frase. En el escenario que mezcla priorización y almacenamiento, el agente puede consultar pedidos y luego buscar en el manual; el límite de ocho iteraciones acota ese ciclo.
2. **Modelo vía OpenRouter, parametrizable.** Separar URL, modelo y clave de la lógica del notebook permite cambiar el proveedor o el modelo sin reescribir las herramientas y mantener las credenciales fuera del código. La temperatura baja favorece respuestas más consistentes en esta demo; como no hay una ejecución real registrada, esa elección aún debe contrastarse con casos de varios pasos.
3. **Herramientas separadas por función.** Dividir lectura, escritura, cálculo y recuperación hace explícito qué operación corresponde a cada necesidad y permite validar sus entradas antes de actuar sobre los datos. Así, una especie o cantidad inválida se informa sin alterar el stock; para escritura con varios cambios relacionados todavía falta una transacción que los agrupe.
4. **SQLite como única persistencia.** Una base local con tablas para inventario, pedidos, movimientos e historial es suficiente para conservar el estado de la demostración y retomarlo al reiniciar el entorno de Colab desde Drive. Mantener una sola persistencia reduce la complejidad del prototipo, aunque no ofrece manejo de concurrencia y la inicialización de la demo puede borrar sus datos.
5. **Memoria corta y memoria larga por sesión.** La ventana de cinco turnos aporta contexto reciente a la conversación, mientras que guardar historial y datos explícitos en SQLite permite reconstruirlo después de limpiar la memoria RAM. Asociar esos datos al session_id evita mezclarlos entre sesiones; el diseño actual supone que se procesa una sesión activa a la vez.
6. **Recuperación con embeddings y respaldo TF-IDF.** Los embeddings permiten buscar fragmentos del manual por similitud de significado, lo que sirve para consultas formuladas con términos distintos a los del texto. TF-IDF mantiene disponible una búsqueda básica cuando no se inicializan los embeddings, aunque compara términos y no garantiza que el fragmento recuperado sea correcto.
7. **ReAct y Plan-and-Execute.** ReAct permite que el agente alterne entre elegir una herramienta y revisar su resultado, algo útil cuando debe consultar datos antes de responder. El modo de planificación agrega pasos sugeridos para solicitudes con varias etapas, mientras el límite de iteraciones restringe el ciclo; la adaptación del plan depende de que el agente interprete los resultados y no está demostrada con un modelo real.
8. **Reglas en el prompt del sistema.** Indicar que las cifras deben venir de herramientas y que los textos del manual y los datos se tratan como información orienta al agente hacia respuestas respaldadas por el estado del sistema. También le pide comunicar errores y atribuir la fuente del manual, aunque estas instrucciones no sustituyen validaciones en código ni una prueba formal contra inyección de instrucciones.

8. Conclusiones y reflexiones individuales

Esteban Pacheco: A nivel personal, el desarrollo de este agente para AgroID Huevos me permitió comprender la complejidad detrás de un sistema basado en LLM y mi mayor aprendizaje fue entender cómo orquestar herramientas (tool calling) mediante LangChain para que el modelo no solo genere texto, sino que interactúe con una base de datos real como SQLite. Este proyecto me permitió ver cómo las soluciones de IA requieren tanto de un buen modelo como de una sólida ingeniería de software detrás.

Ignacio Perez: Consideramos como equipo de desarrollo que esté ejercicio es interesante porque nos brinda un panorama general de cómo se debe construir una solución tecnológica que desde un inicio sea pensado para ser reproducible y escalable en un entorno de trabajo. Nos abre las puertas para entender de mejor manera cómo orquestar reglas de negocio desde la automatización y la seguridad al definir nosotros mismos el alcance y límites del agente. Sin duda es entrar con el pie derecho al mundo de las soluciones IA. Como proyección futura vemos viable la integración de múltiples agentes trabajando en simultáneo además del valor que entregaría escalar este tipo de servicios en la nube para entregar un sistema más complejo, completo y moderno.

9. Declaración de herramientas de IA

Se usaron asistentes de IA en la documentación, revisión y preparación de contenidos del proyecto:

- **OpenAI Codex**: contrastar los archivos con la pauta, ampliar el README, redactar la descripción técnica verificable. No modificó el código fuente.
- **Claude (Anthropic)**, 20 de octubre de 2026: rediseñar el diagrama de arquitectura, crear el diagrama de secuencia, su código Mermaid y el script que genera ambos, actualizar README, y repetir el test simulado en un entorno limpio para registrar versiones. No modificó el código del notebook.

Las reflexiones individuales de la sección 8 quedan a cargo de sus autores.